Detailed Execution Plan: AI-Assisted Macroeconomic Modeling Dashboard

ECO 317 – Assignment #2

---

1. Detailed Step-by-Step Plan

1.1. Step 1: Read the course notes and lock in the exact equations

Goal: Fix the economics before any solver code exists.

Do this:

- Make one shared "model lock-in sheet" with a separate block for Models 1, 2, and 3.
- For each model, write down:
  - Bellman equation,
  - budget or resource constraint,
  - state variables,
  - control variables,
  - shock states,
  - transition matrix,
  - timing convention,
  - variables that must be simulated and plotted.
- Decide coding names now, for example `a`, `a_next`, `k`, `k_next`, `z_idx`, `P`.
- Add one line saying what each solver will return.
- Mark any place where the notes differ from a textbook version.

Example workflow:

- Model 1 block:
  - state = assets and income shock,
  - control = next-period assets,
  - consumption comes from the budget constraint.
- Model 2 block:
  - explicitly write whether output uses current capital or next-period capital.
- Model 3 block:
  - explicitly write whether assets belong in the lecture version.

Deliverable:

- One model lock-in sheet used by the entire team.

Quick check:

- If someone asks "what is the state, control, constraint, and timing?" for any model, the answer should be immediate.

1.2. Step 2: Set up the repository and development environment

Goal: Make the project launch the same way on every laptop.

Do this:

- Create the repository and folder tree.
- Add `requirements.txt` first.
- Create a virtual environment.
- Install packages.
- Make a tiny `app.py` test page to confirm Streamlit launches.
- Check that all teammates can run the app locally.

Files to create immediately:

- `app.py`
- `requirements.txt`
- `README.md`
- `config.py`
- `assets/style.css`
- `models/`, `solvers/`, `simulation/`, `utils/`, `tests/`

Example workflow:

- Use a minimal smoke test first:

```
import streamlit as st

st.title("ECO 317 Project 2")
st.write("Environment test passed.")
```

- Run `streamlit run app.py` before writing any model logic.
- Fix missing-package or Python-version problems now rather than during model development.

Deliverable:

- A clean repo that installs with one command and launches with one command.

Quick check:

- `pip install -r requirements.txt` works.
- `streamlit run app.py` opens in a browser without errors.

1.3. Step 3: Build the project skeleton before filling in model logic

Goal: Make the architecture visible before the hard coding begins.

Do this:

- Create empty modules with docstrings and placeholder functions.
- Define consistent function signatures.
- Decide one solver return structure for all models.
- Decide where parameter dictionaries or dataclasses will live.

Example workflow:

- Put placeholders like this into each model file:

```
def solve_consumption_savings(params):
    """Return value function, policies, grids, and diagnostics."""
    raise NotImplementedError
```

- Decide early that each solver returns keys such as:
  - `value_function`,
  - `policy_indices`,
  - `policy_levels`,
  - `grids`,
  - `diagnostics`.
- This prevents each teammate from inventing a different output format.

Deliverable:

- A modular skeleton where every major task already has a home.

Quick check:

- A new teammate should be able to glance at the file tree and understand where each piece belongs.

1.4. Step 4: Build shared utilities first

Goal: Build the reusable numerical pieces once.

Do this:

- In `utils/grids.py`, write functions to create linear grids and optionally curved grids.
- Add clipping and bounds-check helpers.
- In `utils/markov.py`, write transition-matrix validation and shock-path simulation.
- In `utils/interpolation.py`, wrap `numpy.interp` so simulation and forecasting can evaluate policies off-grid.

- In `simulation/moments.py`, write functions for mean, variance, lag-1 autocorrelation, and correlations.

Example workflow:

- Start the asset or capital grid with something like:

```
grid = np.linspace(x_min, x_max, n_points)
```

- For the Markov chain, create a validator that checks:
  - correct shape,
  - probabilities are nonnegative,
  - each row sums to 1.
- For interpolation, create one helper so all models use the same policy-evaluation rule when the state is off-grid.

Important detail:

- The transition matrix should be validated carefully: rows must sum to 1, probabilities must be nonnegative, and the shock-state ordering must be consistent everywhere.

Deliverable:

- A reusable numerical toolkit used by all three models.

Quick check:

- You can generate a valid grid, simulate a two-state Markov chain, and compute moments on a fake time series.

1.5. Step 5: Implement the generic VFI engine

Goal: Create one Bellman-iteration backbone that all models can use.

Do this:

- Write the main VFI loop in `solvers/vfi.py`.
- Allow the model file to supply model-specific pieces such as utility, feasibility, state transition logic, and candidate controls.
- Track convergence distance at each iteration.
- Return diagnostics such as number of iterations, final error, and whether convergence was achieved.
- Store policy indices so simulation does not need to re-solve the Bellman problem later.

Example workflow:

- Start with:

```
V = np.zeros((n_grid, n_shocks))
for it in range(max_iter):
    V_new = np.empty_like(V)
    policy_idx = np.empty_like(V, dtype=int)
```

- For each state:
  - generate candidate controls,
  - remove infeasible choices,
  - compute current utility,
  - compute expected continuation value,
  - maximize over controls,
  - store the best index.
- At the end of each iteration compute:

```
err = np.max(np.abs(V_new - V))
```

- Stop when `err < tol`.

Important implementation choice:

- Keep the generic solver reusable, but not so abstract that debugging becomes impossible.

Deliverable:

- A reusable solver backbone that every model can call.

Quick check:

- The solver converges on a tiny toy problem with known behavior.

1.6. Step 6: Add hard feasibility checks before any real model runs

Goal: Ensure impossible choices never enter the maximization.

Do this:

- For each model, define a feasibility function.
- Any infeasible choice should be excluded from the maximization.
- Never let negative consumption pass silently.
- Decide whether infeasible values get `-np.inf` utility or are filtered out before maximization.
- Add assertions in testing so infeasible policies are caught immediately.

Example workflow:

- In Model 1, compute:

```
c = (1 + r) * a + y - a_prime
feasible = c > 0
```

- In the labor model also require:
  - $0 \leq L \leq 1$,
  - $1 - L > 0$ when utility needs positive leisure,
  - next-period states stay within bounds.
- Either drop infeasible controls or assign them `-np.inf`.

Deliverable:

- Reliable numerical safeguards.

Quick check:

- With extreme parameter values, the solver should still fail cleanly or exclude bad choices, not produce nonsense.

1.7. Step 7: Implement Model 1 completely before touching Models 2 and 3

Goal: Use Model 1 as the full prototype pipeline.

Do this:

- Create the asset grid.
- Define low and high income states.
- Define the 2×2 Markov transition matrix.
- Implement the utility function, including the log case when `sigma = 1`.
- For each current asset and current income state, evaluate all feasible choices of next-period assets.
- Compute the expected continuation value using the transition matrix.
- Solve for the value function, savings policy, and implied consumption policy.

Example workflow:

- Start with a grid like:

```
a_min, a_max, n_a = 0.0, 20.0, 200
asset_grid = np.linspace(a_min, a_max, n_a)
```

- Define income states such as `y_vals = [y_low, y_high]`.
- Define the transition matrix:

```
P = np.array([[0.9, 0.1],
              [0.1, 0.9]])
```

- For each (a, y):
  - loop through candidate a' values,
  - compute consumption,
  - remove infeasible choices,
  - evaluate Bellman value,
  - save best a' and implied c.

Economic checks:

- Consumption should be positive everywhere.
- Savings should typically rise with assets.
- Higher patience should generally increase saving.
- Higher risk aversion should generally smooth consumption more.

Deliverable:

- One fully solved model with believable policies.

Quick check:

- You can plot $a'(a, y)$ and $c(a, y)$ for both income states and the curves look smooth and economically sensible.

1.8. Step 8: Build simulation and moments for Model 1

Goal: Turn the solved policies into a usable time series.

Do this:

- Choose an initial asset level and initial shock state.
- Simulate a shock path using the Markov chain.
- Use the saved policy functions each period.
- Interpolate when the current state falls between grid points.
- Store the full time paths for assets, consumption, and income.
- Convert the arrays into a DataFrame.
- Compute mean, variance, lag-1 autocorrelation, and correlation with income.

Example workflow:

- Fix:

  - $T = 100$ or 200,
  - a_0,
  - initial shock state,
  - random seed.
- At each period:
  - read current state,
  - interpolate the policy if the asset level is off-grid,
  - compute next assets,
  - recover consumption from the budget constraint,
  - store assets, consumption, and income.
- Build a DataFrame with columns like `assets`, `consumption`, `income`.

Important detail:

- Do not recompute the maximization during simulation. Simulation should only read the solved policy functions.

Deliverable:

- A complete Model 1 pipeline from solver to time series to moments.

Quick check:

- With the same random seed, the simulation should be reproducible.

1.9. Step 9: Build forecast logic for Model 1 using the solved policy functions

Goal: Forecast with policy rules instead of trend fitting.

Do this:

- Write a forecast function that accepts the current state and a user-chosen future shock path.
- Use the actual policy rules to update the state period by period.
- Generate forecast paths for consumption, savings or assets, and income.
- Make the horizon configurable.
- Keep the forecast logic separate from stochastic simulation, since here the future shock path is user-controlled.

Example workflow:

- Suppose the current state is "assets = 4.2, shock = low income."
- Suppose the chosen future shocks are `[high, high, low, high]`.
- For each forecast step:
  - evaluate the saved policy,

  - update assets,
  - move to the next user-chosen shock,
  - store the forecasted variables.
- Keep the forecast arrays separate from the stochastic simulation arrays.

Deliverable:

- Forecast series that are genuinely implied by the solved model.

Quick check:

- If the user chooses a sequence of high shocks, the forecast should reflect that path in a way consistent with the model's policy functions.

1.10. Step 10: Implement Model 2 only after a timing checkpoint

Goal: Build the Robinson Crusoe model without a timing error.

Do this:

- Re-open the notes and verify the exact capital-output timing.
- Define the capital grid and TFP shock states.
- Implement the transition matrix for TFP.
- Write production and depreciation logic.
- Solve for the value function, next-period capital policy, and implied consumption policy.
- Add implied output and investment calculations for simulation and plots.

Example workflow:

- Write a comment at the top of the file stating the timing convention taken from the notes.
- Start with:

```
k_grid = np.linspace(k_min, k_max, n_k)
z_vals = np.array([z_low, z_high])
```

- For each (k, z):
  - loop over candidate k',
  - compute output,
  - compute consumption,
  - exclude infeasible choices,
  - maximize Bellman value.
- Save derived quantities such as output and investment for simulation plots.

Economic checks:

- Output should be higher in the good TFP state.
- Capital policy should usually respond positively to better productivity.
- Consumption and investment should behave sensibly when TFP changes.

Deliverable:

- A solved stochastic growth-style model with capital accumulation.

Quick check:

- The model should produce reasonable paths for capital, output, consumption, and investment.

1.11. Step 11: Implement Model 3 with labor-leisure trade-offs and confirm whether assets belong

Goal: Build the labor model exactly as the lecture version intends.

Do this:

- Confirm from the notes whether the state includes assets or just the wage shock and labor choice.
- Define wage states and the wage transition matrix.
- Define the labor grid on $[0, 1]$.
- If assets are included, jointly search over labor and next-period assets.
- If assets are not included, solve the simpler static or dynamic labor-choice version from the notes.
- Recover labor, consumption, and savings policies if relevant.
- Simulate labor, wage, and consumption paths.
- Build at least one static intuition graph, such as labor against wage or labor against the preference parameter.

Example workflow:

- Start with a labor grid:

```
labor_grid = np.linspace(0.0, 1.0, 51)
```

- If assets are included:
  - loop over a',
  - loop over L,
  - compute c and leisure $1 - L$,
  - keep only feasible points.
- If assets are not included:

  - do not force an asset dimension into the model,
  - solve the simpler lecture version instead.
- Save policies for labor, consumption, and assets if applicable.

Important detail:

- Make sure leisure is coded as $1 - L$ and stays positive.

Deliverable:

- A labor-supply module that can explain both substitution and wealth-effect intuition.

Quick check:

- Labor always lies in $[0, 1]$, and the resulting plots are economically interpretable.

1.12. Step 12: Unify simulation, forecasting, and moments across all three models

Goal: Give the app one common interface across all models.

Do this:

- Make sure each model returns results in a comparable structure.
- Standardize naming for states, controls, and series.
- Have one shared simulation interface and one shared forecast interface.
- Build a shared moments table generator that can adapt to different variables by model.
- Decide which series are the key variables for each model.

Example workflow:

- Make each model return a dictionary with the same top-level keys:
  - `solution`,
  - `simulation`,
  - `forecast`,
  - `moments`,
  - `plots_data`.
- Use model-specific variable names inside a common format.
- This lets `app.py` render different models without branching everywhere.

Deliverable:

- A common API that the Streamlit app can call without model-specific hacks scattered everywhere.

Quick check:

- Swapping from Model 1 to Model 2 in the app should mostly change the data, not the architecture.

1.13. Step 13: Build the Streamlit dashboard only after one full model pipeline is stable

Goal: Put the interface on top of working economics rather than unfinished economics.

Do this:

- Load CSS from `assets/style.css`.
- Add sidebar navigation between the three models.
- Add model-specific parameter controls in the sidebar.
- Add controls for transition probabilities, simulation horizon, and forecast horizon.
- Solve the selected model using cached functions.
- Run simulation and forecasting for the selected model.
- Display the outputs in a consistent page structure.

Example workflow:

- Sidebar controls:
  - model selector,
  - parameters,
  - transition probabilities,
  - simulation horizon,
  - forecast horizon.
- Main-page order:
  - title and model description,
  - equations used,
  - policy function plot,
  - simulated time-series plot,
  - forecast plot,
  - comparative-statics graph,
  - moments table,
  - static intuition section,
  - dynamic summary text.
- Cache the expensive solver call so minor UI interactions do not trigger full re-solves unnecessarily.

Deliverable:

- One app that makes the three models feel like one project rather than three unrelated scripts.

Quick check:

- A user can switch models without the app breaking or looking inconsistent.

1.14. Step 14: Add comparative statics and dynamic written summaries

Goal: Make the app explain what changes when parameters change.

Do this:

- Add a baseline-versus-alternative parameter comparison for at least one model.
- Good candidates are higher `beta`, higher `sigma`, or different wage or TFP persistence.
- Plot the two policy functions on the same figure or in side-by-side panels.
- Write summary functions that read the moments and generate deterministic interpretation text.

Example workflow:

- Solve one baseline calibration and one alternative calibration.
- Example:
  - baseline $\beta = 0.95$,
  - alternative $\beta = 0.98$.
- Plot both policy functions together and label them clearly.
- Generate summary text with templates such as:

```
summary = (
    f"Mean consumption is {mean_c:.2f}. "
    f"When beta rises, the policy shifts toward more saving."
)
```

- Keep the runtime text deterministic and tied to computed results.

A good summary should do four things:

- identify the variable,
- state what changed,
- connect it to economic intuition,
- avoid claiming more than the computed results show.

Deliverable:

- Visual and written interpretation of live results.

Quick check:

- The text below the charts changes when the user changes parameters, and the wording reflects the actual computed moments.

1.15. Step 15: Add styling, caching, and performance protection

Goal: Make the app pleasant to demo and fast enough to use live.

Do this:

- Create a dark or navy background and bright white content cards.
- Use a robust serif font stack if local Garamond is unavailable.
- Wrap expensive solver functions in Streamlit caching.
- Choose default grid sizes and horizons that update within a few seconds.
- Keep plot labels clean and readable.
- Make sure the page layout is visually consistent across models.

Example workflow:

- Put visual rules in `assets/style.css`.
- Start with conservative defaults such as:
  - 150–250 grid points,
  - 100–200 simulated periods,
  - cached solver output keyed by parameter values.
- Time common interactions such as changing β or forecast length.
- If one slider creates a long delay, lower the default grid density or cache more aggressively.

Deliverable:

- A dashboard that looks intentional and behaves smoothly.

Quick check:

- Standard slider changes should update quickly on an ordinary laptop.

1.16. Step 16: Test mathematical correctness, numerical stability, and UI behavior

Goal: Convert "works on my machine" into "safe to demo live."

Do this:

- Add tests to verify positive consumption, valid policy bounds, and convergence.
- Check for NaNs, infinities, or jagged policy plots.
- Verify that interpolation never pushes states outside valid ranges.
- Confirm that comparative statics move in sensible directions.
- Test every page, slider, and forecast control in the dashboard.
- Compare the moments table to direct calculations from the simulated data.

Example workflow:

- Add tests such as:
  - "all simulated consumption values are positive,"
  - "all labor choices lie in $[0, 1]$,"
  - "solver converges before max iterations."
- Inspect plots visually for jagged or discontinuous policies.
- Compare dashboard moments to direct `numpy` or `pandas` calculations from the same arrays.
- Click through all pages and try awkward inputs intentionally.

Deliverable:

- A tested project rather than a fragile one.

Quick check:

- Every model passes a basic suite of solver, simulation, and UI sanity checks.

1.17. Step 17: Keep a prompt log and explainable comments throughout development

Goal: Be ready to explain both the code and the AI workflow.

Do this:

- Keep a simple prompt log with one entry per major component.
- For each entry, record the prompt, what file it helped create, and how the output was checked or corrected.
- Add comments explaining the economics, not just the syntax.
- Mark the key lines in the code that implement constraints, Bellman updates, and policy extraction.

Example workflow:

- A prompt-log entry should contain:
  - prompt used,
  - file affected,
  - what part of the output was kept,
  - what had to be corrected,
  - how the economics were verified.
- In code comments, prefer comments like "Compute expected continuation value over next-period shocks" instead of comments that only restate syntax.

Deliverable:

- Evidence that AI was used responsibly and verified.

Quick check:

- If the professor points to a function, someone in the group can explain both what it does and how it was validated.

1.18. Step 18: Rehearse the live demo exactly as it will happen

Goal: Practice the actual grading situation, not just the software.

Do this:

- Start from a fresh terminal and run the full install and launch sequence.
- Practice switching between all three models.
- Practice changing parameters and narrating the economic effect.
- Practice explaining why VFI is used, what the Markov chain does, why interpolation is needed, and why caching matters.
- Practice answering "show me where this is in the code."

Example workflow:

- Rehearsal round 1:
  - one teammate runs the terminal,
  - one teammate acts as the professor,
  - one teammate records confusion points.
- Rehearsal round 2:
  - switch presenters,
  - ask code questions like "where is the Bellman update?" and "where is interpolation used?"
- Rehearsal round 3:
  - rapidly switch models and change parameters to test robustness.

Deliverable:

- A group that can both run and defend the project.

Quick check:

- Any teammate can handle a random question without needing someone else to rescue the explanation.

2. Final Order of Execution

If the group wants the shortest operational version, follow this exact sequence:

1. Lock equations from the course notes.
2. Set up the repository and environment.
3. Create the file skeleton.
4. Build shared utilities.

5. Build the generic VFI solver.
6. Add feasibility checks.
7. Finish Model 1 completely.
8. Add simulation and moments for Model 1.
9. Add forecasting for Model 1.
10. Implement Model 2 after timing verification.
11. Implement Model 3 after confirming whether assets are included.
12. Standardize simulation, forecast, and moments across all models.
13. Build the Streamlit dashboard.
14. Add comparative statics and dynamic summaries.
15. Add styling, caching, and performance tuning.
16. Run mathematical, numerical, and UI tests.
17. Maintain the prompt log and comments.
18. Rehearse the live demo from scratch.

Bottom line. The project should be built in a way that makes the economics correct first, the numerical methods trustworthy second, and the dashboard polished third. A clean live demo is important, but the strongest defense comes from having a team that can explain exactly why the model equations, policy functions, forecasts, and summaries are correct.